



Entity Framework Integration

John Lyndon Philip Planas

Software Developer 2, Open iT



Agenda

1. Introductions
2. Setting up the environment
3. Integration Proper
 - What is Entity Framework?
 - How to integrate your code to database?



Setting up the Environment

**Check if the necessary tools are available
in your system**



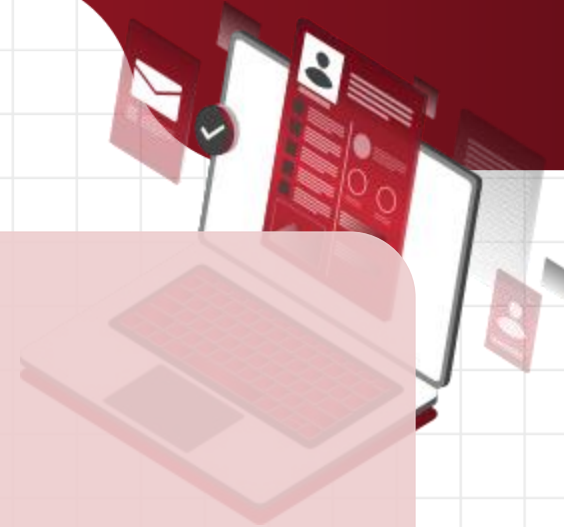
Setting up the Environment

VS Code

- Code Editor
- Where we write our C# code.

.NET SDK

- Code Compiler
- Allows to create and execute C# applications.
- Try:
 - `dotnet --version`



Setting up the Environment



PostgreSQL

Database system (data storage)
Where data is permanently stored.



pgAdmin

Dashboard or Control Panel of
PostgreSQL
A tool for managing and ease of
configuration for the database system.

Quick Question



What do you think happens if we remove the database?



Can an application work without storing data?

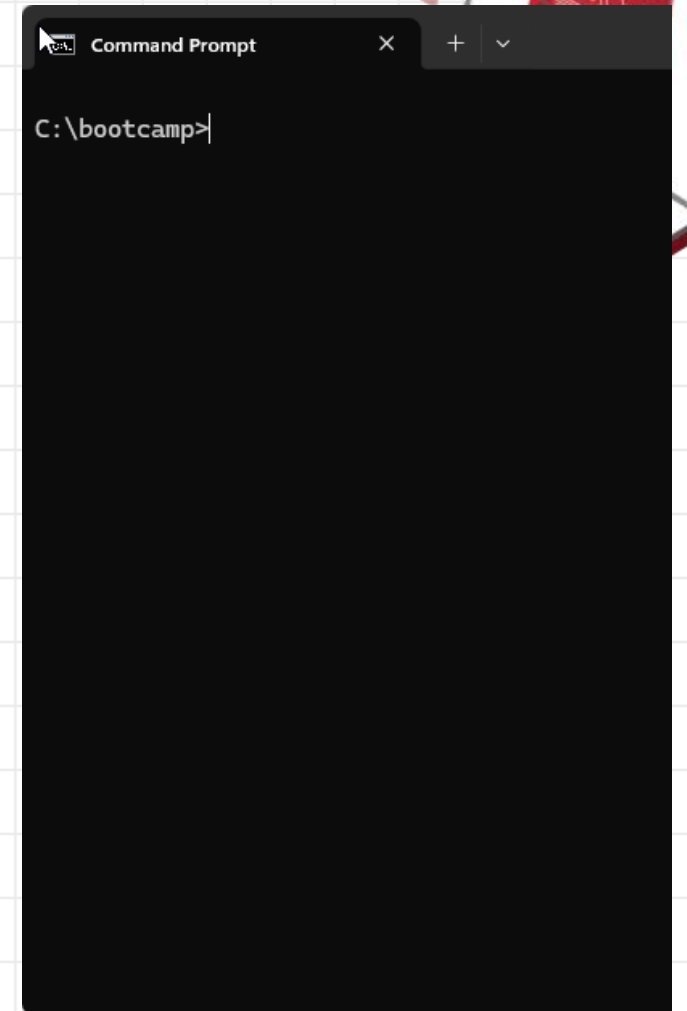
Setting up the .NET Project

Setup all the necessary packages for .NET Project



Creating the .NET Project

- Create a folder named **bootcamp** in **C:**.
 - C:\bootcamp
- Open a command line on the folder.



Creating the .NET Project

- Run the command:
 - dotnet new console –n BootcampEF
- This will create a project named **BootcampEF**.



Part	Meaning
dotnet	.NET CLI
new	Create
Console	Console application
-n	Name
BootcampEF	Project Name

Creating the .NET Project



Open the project in **VS Code** by running the following command:

```
cd BootcampEF  
code .
```



In **VS Code**, see what files are automatically created.

Installing Packages

- **EF Core Main Package** (Entity Framework)
 - An **ORM (Object-Relational Mapping) package** is a library that lets you interact with a database using **objects and methods** instead of writing raw SQL queries. It maps database tables to programming language classes, and rows to objects, making database operations more intuitive and less error-prone.
 - In **VS Code**, open a new Terminal, then run the command:
 - `dotnet add package Microsoft.EntityFrameworkCore`



Installing Packages

- **Design Package**

- This package helps us use migrations. a **migration** is a way to **track and apply changes to your database schema** over time using code.
- Run the command:
 - `dotnet add package Microsoft.EntityFrameworkCore.Design`



Installing Packages

- **PostgreSQL Provider**

- This package allows EF Core to communicate specifically with **PostgreSQL**.
- Run the command:
 - `dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL`



Integration Proper

Connect your code to the database

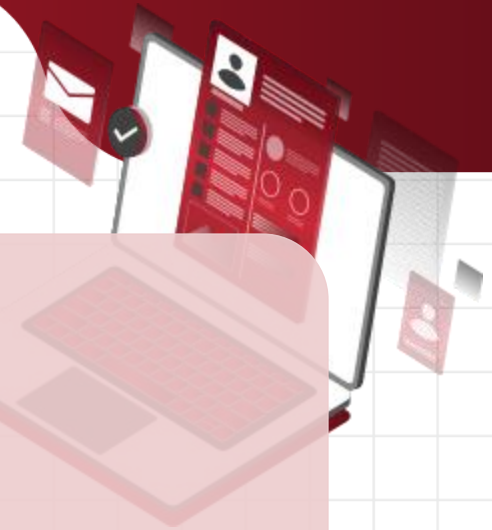


Creating the Model

- **A Model is:**
 - A **C# class** that represents the structure of data in the database.
 - “A model is like a blueprint or template for our data.”
 - In **Entity Framework Core**, models represent **database tables**.”



Creating the Model



In **VS Code**,
create a new file.
Name the file as:

- **Student.cs**

Add the
following code:

- ```
public class Student
{
 public int Id { get; set; }

 public string Name { get; set; }

 public string Course { get; set; }
}
```

# The Model

- **public class Student**

- This creates a class named Student.
- EntityFramework will convert this class into a database table.

- **public int Id**

- This becomes the **Primary Key** in the database table.
- **EF Core** automatically recognizes a property named **Id** as the **Primary Key**.



# The Model

- **public string Name**
  - This becomes a column in the database named **Name**.
- **public string Course**
  - This becomes a column in the database named **Course**.



| C# Object | Database       |
|-----------|----------------|
| Student   | Student table  |
| Id        | Id column (PK) |
| Name      | name column    |
| Course    | course column  |

# Quick Game – Coding Relay

- **Rules of the game:**

- We need **6 players** for this game.
- Each player will be adding code lines one-by-one to complete the requirement.
- The next player can do **only 1 of the two things**:
  - Add a new line of code.
  - Modify **ONLY 1 line** to correct the code.
- Please code properly. **One line per player ONLY!**
- **The winning team will receive a PRIZE! 😊**



# Quick Game – Coding Relay

- **Create a model:**
  - **Name:** Contacts
  - **Properties:**
    - **Id:** number
    - **Name:** text
    - **Age:** number
    - **Gender:** text
    - **Address:** text
    - **Contact\_Number:** text
    - **Email:** text



# Creating DbContext

- **DbContext** is the heart of Entity Framework.
  - Manages database connection
  - Handles queries
  - Saves changes



# Creating DbContext

In **VS Code**, create a new file. Name the file as:

- **StudentContext.cs**

Add the following code:

```
using Microsoft.EntityFrameworkCore;
public class StudentContext : DbContext
{
 public DbSet<Student> Students { get; set; }
 protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
 {
 optionsBuilder.UseNpgsql(
 "Host=localhost;Port=5432;Database=bootcamp;Username=postgres;Password=pa$$w0Rd"
);
 }
}
```



# The DbContext

- **DbSet<Student>**
  - Represent the **Student table** or the **Student model**.
- **UseNpgsql()**
  - Connect EF Core to **PostgreSQL**.
- **Connection String**
  - A string containing necessary parameters needed to connect to the PostgreSQL sever.



# Connection String

- **Host=localhost;Port=5432;Database=bootcamp;User name=postgres;Password=pa\$\$w0Rd**



| Parameter Name | Meaning                                       |
|----------------|-----------------------------------------------|
| Host           | Database server name (ip address or hostname) |
| Port           | PostgreSQL port (commonly <b>5432</b> )       |
| Database       | Database name                                 |
| Username       | Login username                                |
| Password       | Login user password                           |

# Creating Migrations

- **Migrations** allow EF Core to create database structures automatically.
- A feature in Entity Framework Core that updates the database structure based on changes in our C# models.



# Installing EF Migration Tool

- To install the migration commands, run the command:
  - `dotnet tool install --global dotnet-ef`



# Adding Migration

- To add migrations, run the command:
  - dotnet ef migrations add InitialCreate
- **InitialCreate** – name of the migration
- This reads the created models.
- Generates migration files



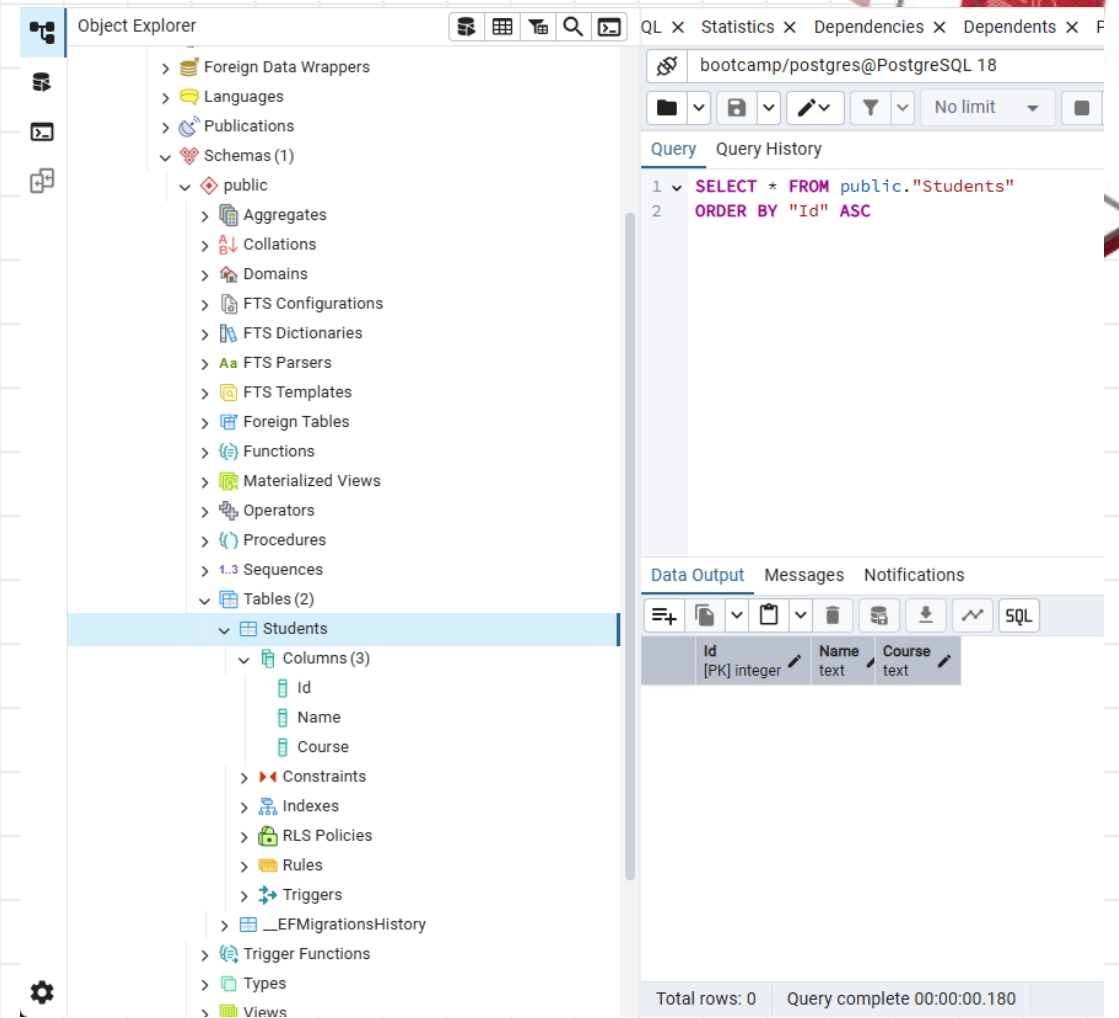
# Applying Migrations

- To apply the migrations to the database, run the command:
  - dotnet ef database update
    - This applies the changes to the database based on the created migrations.



# Refresh the Database

- Open the **pgAdmin** and refresh the bootcampdatabase.
  - See that the Student table is created with the configured columns.



# CRUD Operations

**Four basic database operations.**



# CRUD Operations

| Letter | Meaning |
|--------|---------|
| C      | Create  |
| R      | Read    |
| U      | Update  |
| D      | Delete  |



# Create

- Add new record in the database
- In VS Code, open **Program.cs** and overwrite the content with the following C# Code.
- Once saved, in the **Terminal**, run the command:
  - **dotnet run**



# Create

## C# Code

```
using System;

using var context = new StudentContext();

var student = new Student
{
 Name = "John",
 Course = "IT"
};

context.Students.Add(student);

context.SaveChanges();

Console.WriteLine("Student Added!");
```

## Explanation

Creates a connection object to the database using **StudentContext**.

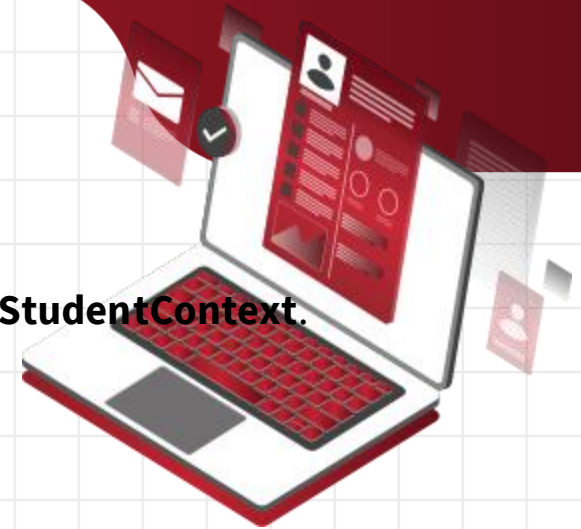
Creates a new **Student** object in C#.

```
{
 Sets the Name property of the student.
 Sets the Course property of the student.
};
```

Prepare the record to be inserted into the **Students** table.

Saves the new student into **PostgreSQL**.

Displays a message in the terminal.



# Read

- Read records in the database
- Still in **Program.cs**, add the following code.
- Once saved, in the **Terminal**, run the command:
  - **dotnet run**



# Read

## C# Code

```
var students = context.Students.ToList();

Console.WriteLine("\nStudent List:");

foreach (var s in students)
{
 Console.WriteLine($"{s.Id} - {s.Name} - {s.Course}");
}
```

## Explanation

Retrieves all records from PostgreSQL.

Displays a message in the terminal.

Loops through each record in the table and displays each record.



# Read

```
Program.cs
1 using System;
2
3 using var context = new StudentContext();
4
5 Console.WriteLine("Connected to PostgreSQL!");
6
7 var student = new Student
8 {
9 Name = "John",
10 Course = "IT"
11 };
12
13 context.Students.Add(student);
14
15 context.SaveChanges();
16
17 Console.WriteLine("Student Added!");
18
19 var students = context.Students.ToList();
20
21 Console.WriteLine("\nStudent List:");
22
23 foreach (var s in students)
24 {
25 Console.WriteLine($"{s.Id} - {s.Name} - {s.Course}");
26 }
27
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\bootcamp\BootCampEF> dotnet run  
Connected to PostgreSQL!  
Student Added!

Student List:  
1 - John - IT

PS C:\bootcamp\BootCampEF>



# Update

- Change or modify a record in the database
- Still in **Program.cs**, add the following code.
- Once saved, in the **Terminal**, run the command:
  - **dotnet run**



# Update

## C# Code

```
var firstStudent =
context.Students.FirstOrDefault();
```

```
if (firstStudent != null)
{
 firstStudent.Name = "Updated John";
```

```
context.SaveChanges();
```

```
Console.WriteLine("\nStudent Updated!");
}
```

## Explanation

Gets the first record in the Students table.

Check if any data is retrieved

```
{
 Assign the Name property of the retrieved record to 'Updated
 John'.
```

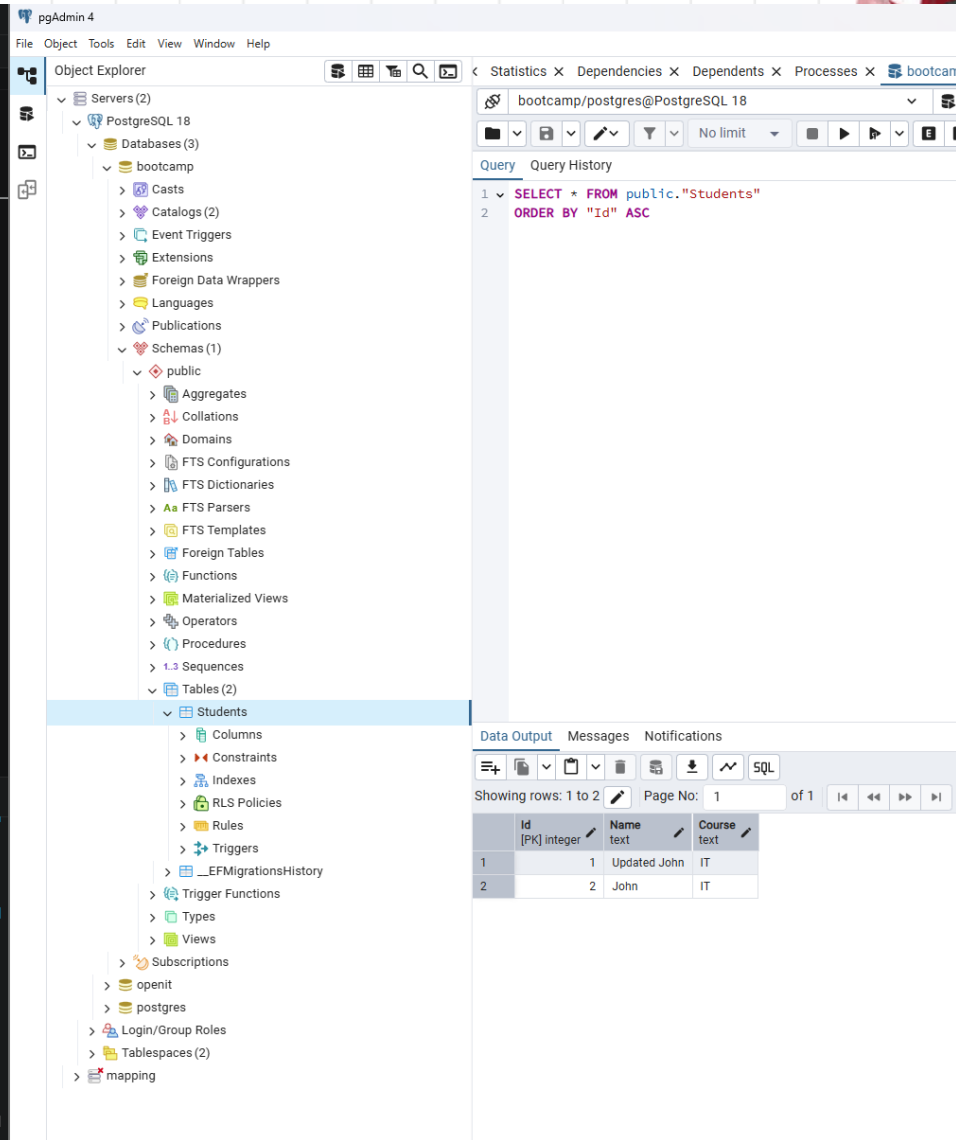
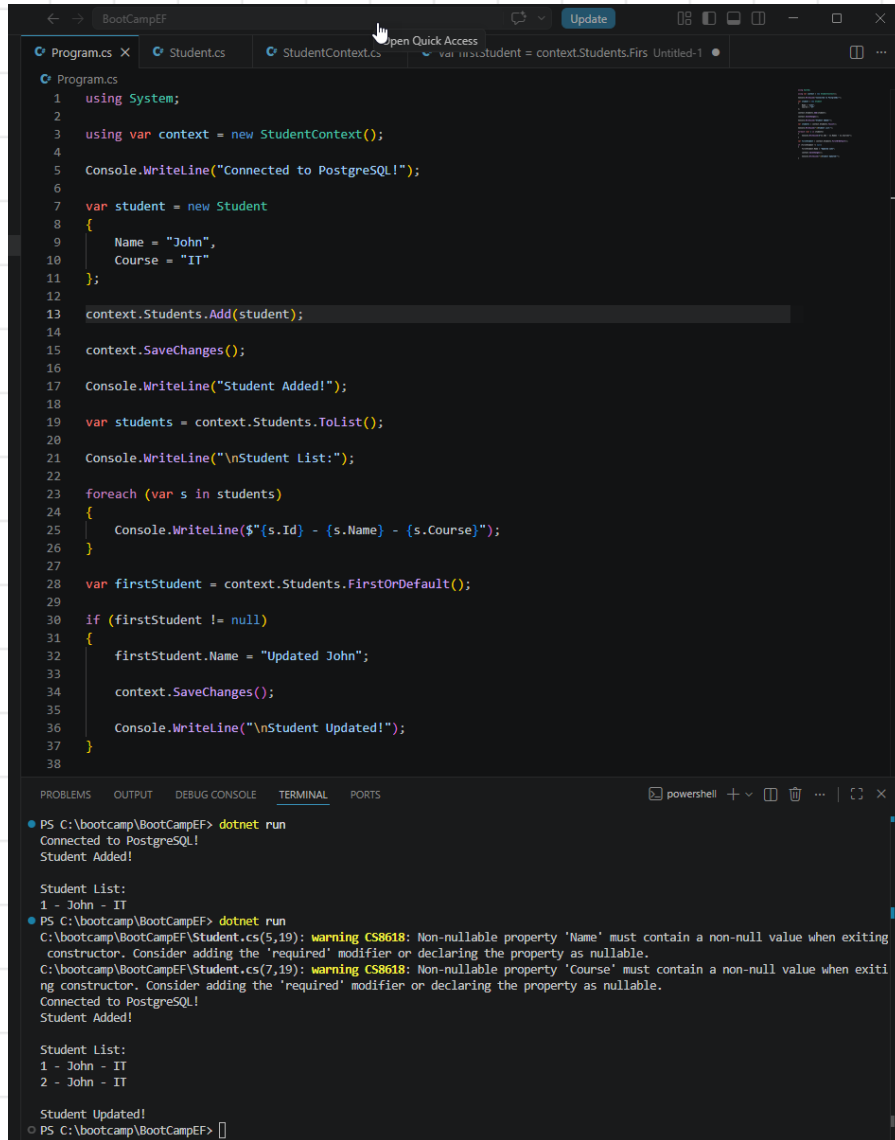
Save the changes to the database.

Displays a message in the terminal.

```
}
```



# Update



# Delete

- Remove a record in the database
- Still in **Program.cs**, comment out the **Add** and **Update** codes.
- Add the following code before the **Read** code.
- Once saved, in the **Terminal**, run the command:
  - **dotnet run**



# Delete

## C# Code

```
var deleteStudent =
context.Students.FirstOrDefault();

if (deleteStudent != null)
{
 context.Students.Remove(deleteStudent);

 context.SaveChanges();

 Console.WriteLine("\nStudent Deleted!");
}
```

## Explanation

Get the first record in the Students table.

Check if there is any data retrieved  
{

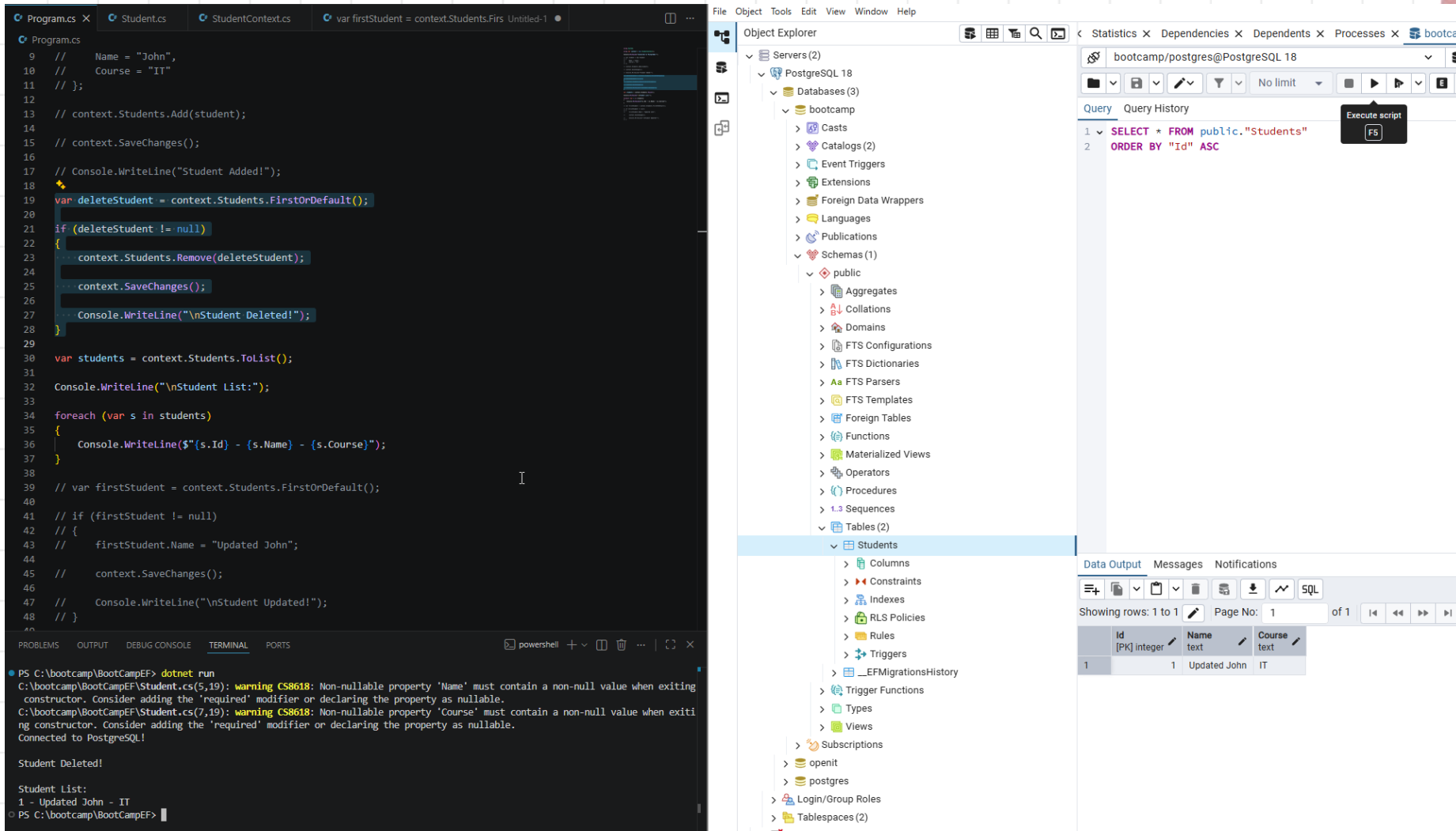
Prepares the record to be removed.

Remove the record and save the changes.

Displays a message in the terminal.  
}



# Delete



# Relationships

## Table connections



# Relationships

- This connects tables together.
- This organizes connected data efficiently.
- Without relationships:
  - Data becomes duplicated
  - Harder to manage
  - inconsistent

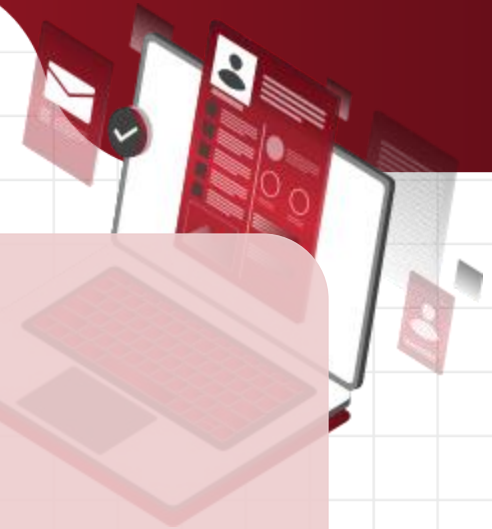


# Types of Relationships

- **One-to-One**
  - Example: One student is to one student number.
- **One-to-Many**
  - Example: One student is to many courses.
- **Many-to-Many**
  - Example: Many students is to many subjects.



# Relationships



In **VS Code**,  
create a new file.  
Name the file as:

- **Course.cs**

Add the  
following code:

```
• public class Course
{
 public int Id { get; set; }

 public string Name { get; set; }

}
```

# Relationships

In VS Code,  
open the  
following file:

• **Student.cs**

Overwrite the  
content with :

```
• public class Student
{
 public int Id { get; set; }
 public string Name { get; set; }
 public int CourseId { get; set; }
 public Course Course { get; set; }
}
```

# Student Model

- **public int CourseId**
  - This serve as the **Foreign Key**.
    - stores the ID of a related table.
- **public Course Course**
  - This serves as a **Navigation Property**.
    - Which lets us access the related Course data.



# Relationships

In VS Code,  
update the  
following file:

- **Course.cs**

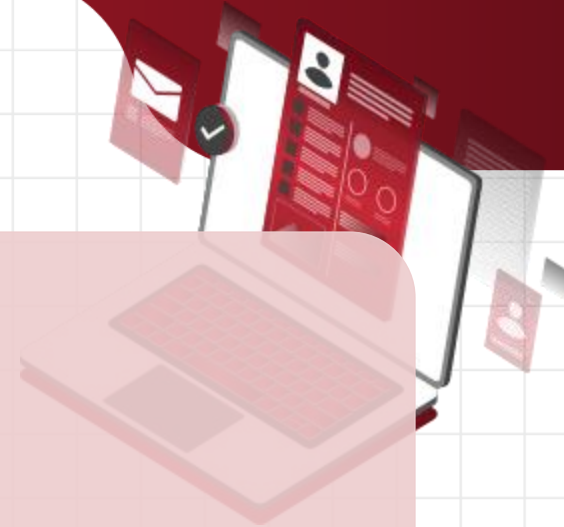
Overwrite the  
file with:

- ```
public class Course
{
    public int Id { get; set; }

    public string Name { get; set; }

    public List<Student> Students { get; set; }

}
```



Course Model

- **public List<Student> Students**
 - Relationship of **Course** and **Students** tables.
 - One Course can contain Many students. (**One-to-Many**)



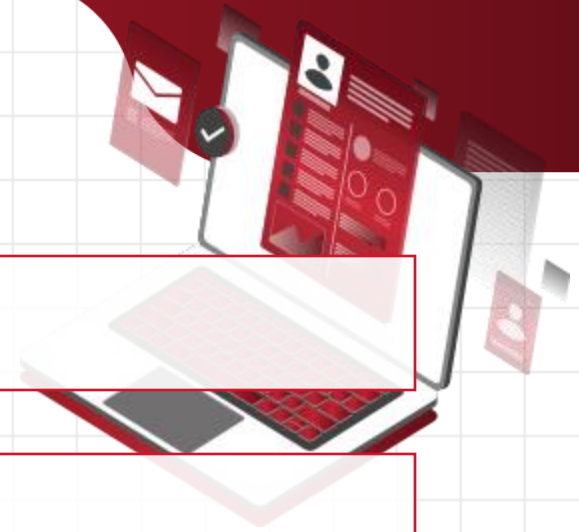
Update DbContext

In **VS Code**, open the following file:

- **StudentContext.cs**

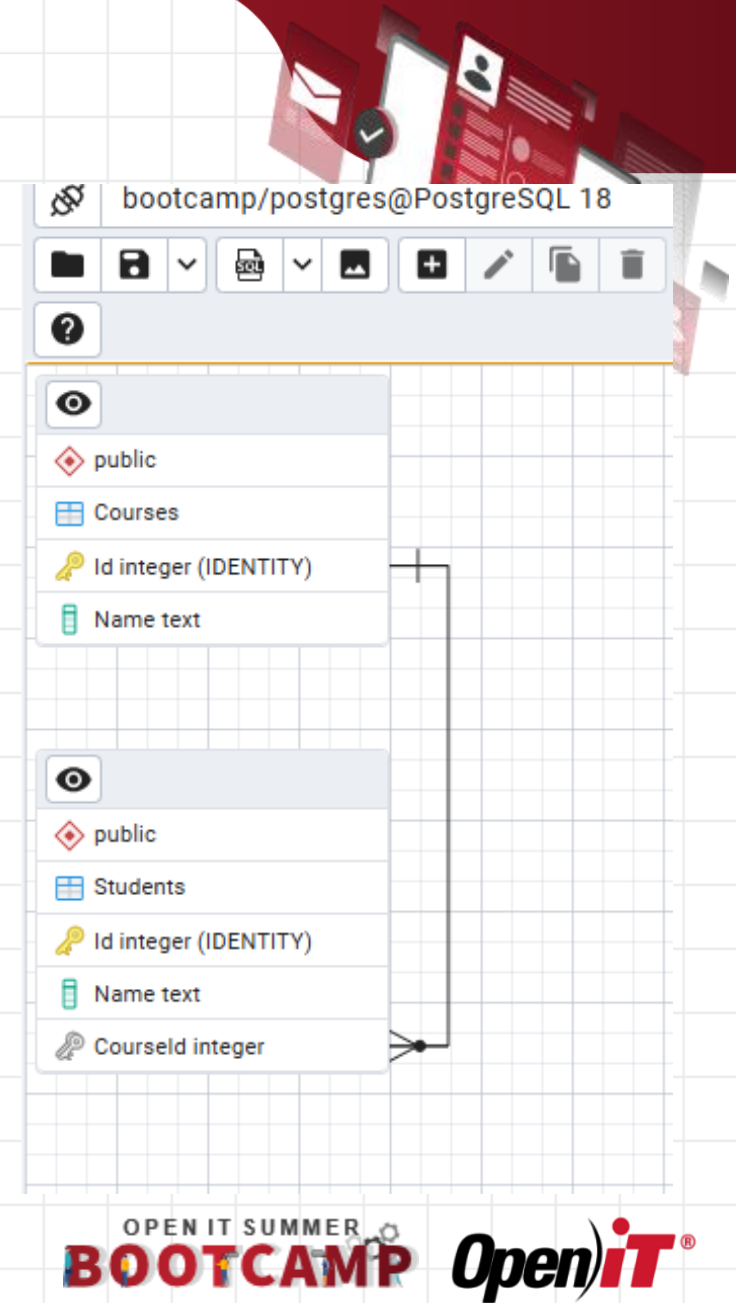
Overwrite the code with:

```
using Microsoft.EntityFrameworkCore;
public class StudentContext : DbContext
{
    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseNpgsql(
            "Host=localhost;Port=5432;Database=school_db;Username=postgres;Password=yourpassword"
        );
    }
}
```



Applying Course Relationship

- **Please delete data in the Students table first!**
 - In **pgAdmin** run the query: **DELETE FROM public."Students"**
- **Create a migration, run the command:**
 - dotnet ef migrations add AddCourseRelationship
- **Update the database, run the command:**
 - dotnet ef database update



Add a Course

Add Relationship to Student

C# Code

```
using System;
using var context = new StudentContext();

var course = new Course
{
    Name = "IT",
};

context.Courses.Add(course);

context.SaveChanges();
```

```
var student = new Student
{
    Name = "John",
    CourseId = course.Id
};

context.Students.Add(student);

context.SaveChanges();
```



Retrieve Related Data

C# Code

```
using System;
using Microsoft.EntityFrameworkCore;
using var context = new StudentContext();

var students = context.Students
    .Include(s => s.Course)
    .ToList();

foreach (var s in students)
{
    Console.WriteLine($"{s.Name} - {s.Course.Name}");
}
```

Explanation

Include - loads the related data

With Include – Course data loads too

Without Include – only Student data loads



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\bootcamp\BootCampEF> dotnet run
  Connected to PostgreSQL!
  John - IT
○ PS C:\bootcamp\BootCampEF> |
```

Insert Multiple Students

C# Code

```
using System;
using Microsoft.EntityFrameworkCore;
using var context = new StudentContext();

var course = new Course
{
    Name = "IS",
    Students = new List<Student>
    {
        new Student { Name = "Johnson" },
        new Student { Name = "Anna" },
        new Student { Name = "Mark" }
    }
};

context.Courses.Add(course);
context.SaveChanges();

Console.WriteLine("Course and students added!");
```

Explanation

Create an object **Course** with **IT** as **Name** and include the list of students.



Quick Activity

- Please insert a Course
 - **Name:** CS
 - **Students:**
 - Mary
 - Joshua
 - Pia



EF Core Relationship Components

Component	Purpose
Foreign Key	Connect tables
Navigation Property	Access related database
Include()	Load related objects
Migration	Created DB relationships



Entity Framework Core relationships allow us to connect tables using **C# classes**, while **PostgreSQL** stores and enforces those relationships behind the scenes.”

Generated Query String

Display and investigate the generated query string



Generated Query String

C# Code

```
using System;  
using Microsoft.EntityFrameworkCore;  
using var context = new StudentContext();
```

```
var students = context.Students  
    .Include(s => s.Course)  
    .ToList();
```

```
foreach (var s in students)  
{  
    Console.WriteLine($"{s.Name} -  
    {s.Course.Name}");  
}
```

```
Console.WriteLine(  
    context.Students  
        .Include(s => s.Course)  
        .ToString()  
);
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● PS C:\bootcamp\BootCampEF> dotnet run  
John - IT  
Anna - IT  
Mark - IT  
SELECT s."Id", s."CourseId", s."Name", c."Id", c."Name"  
FROM "Students" AS s  
INNER JOIN "Courses" AS c ON s."CourseId" = c."Id"  
○ PS C:\bootcamp\BootCampEF> |
```

Sorting Data

Organizing data retrieval based on a certain attribute



Sorting Data

- **var students = context.Students.ToList();**
 - This retrieves data and convert it to list but the order of the data is unpredictable.
 - Databases do **NOT** guarantee order unless we **explicitly sort**.



Method	Purpose
OrderBy()	Ascending
OrderByDescending()	Descending
ThenBy()	Secondary sort
ThenByDescending()	Secondary descending

Sorting Data

C# Code

```
using System;  
using Microsoft.EntityFrameworkCore;  
using var context = new StudentContext();  
  
var students = context.Students  
    .OrderBy(s => s.Name)  
    .ToList();  
  
foreach (var s in students)  
{  
    Console.WriteLine($"{s.Name} - {s.Name}");  
}
```

Explanation

Sorts the data in **ascending order**
Sort using the **Name** property.



Difference between OrderBy() and ThenBy()

- **ThenBy()**

- Only works **AFTER** OrderBy() or OrderByDescending().
- It sorts data **after the first sorting condition** from OrderBy().



ThenBy()

C# Code

```
using System;
using Microsoft.EntityFrameworkCore;
using var context = new StudentContext();

var students = context.Students
    .Include(s => s.Course)
    .OrderBy(s => s.Course.Name)
    .ThenBy(s => s.Name)
    .ToList();

foreach (var s in students)
{
    Console.WriteLine($"{s.Course.Name} - {s.
Name}");
}
```

Explanation

Sorts the data in **ascending order**

Sort using the **Course Name** property, then **Name**.



Setting Data Annotations

Property attributes for restrictions



Data Annotations

- Data Annotations are attributes we place above properties to:
 - validate data
 - control database behavior
 - configure table columns
- Data annotations are instructions we add to our model properties to control how the database behaves.



Most Common Data Annotations

Annotation	Purpose
[Required]	Cannot be NULL
[MaxLength(<number>)]	Maximum character
[MinLength<number>]	Minimum character
[Range(<number>, <number>)]	Numeric limits
[EmailAddress]	Email validation
[Phone]	Phone validation
[Key]	Primary Key
[ForeignKey(<table>)]	Relationship
[NotMapped]	Ignore column
[Column]	Customize column
[Table]	Customize table name
[StringLength]	Min + Max length



Data Annotations Example

using System.ComponentModel.DataAnnotations;

```
public class Student
{
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Name { get; set; }

    public int CourseId { get; set; }

    public Course Course { get; set; }
}
```

Explanation

Required namespace

Set it to **NOT NULL**.

Set **maximum length** to 50.

To apply, please run the command:

dotnet ef migrations add AddAnnotationsName
dotnet ef database update



Conditional Query

Filter data based on conditions



Conditional Query

Id	Name	Course
1	John	IT
2	Maria	CS
3	Anna	IT



- **SQL**

- SELECT * FROM “Students” WHERE “Course” = “IT”

- **C# (EF)**

- var students = context.Students
 .Where(s => s.Course == "IT")
 .ToList();

Conditional Query (Related table)

```
using System;  
using Microsoft.EntityFrameworkCore;  
using var context = new StudentContext();
```

```
var students = context.Students  
    .Include(s => s.Course)  
    .Where(s => s.Course.Name == "CS")  
    .ToList();
```

```
foreach (var s in students)  
{  
    Console.WriteLine($"{s.Course.Name} - {s.  
Name}");  
}
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
● PS C:\bootcamp\BootCampEF> dotnet run  
CS - Pia  
CS - Joshua  
CS - Mary  
○ PS C:\bootcamp\BootCampEF> █
```

Activity

- You have created the tables and inserted and retrieve data manually on your activity in **PostgreSQL** topic.
- Action Points:
 - Convert the schema into a models. Please include proper relationships.
 - Create a function that will emulate the Part 2 (Seed Data)
 - Create a function that will retrieve Student Enrolment data.
 - The programs should start with a menu and the user can select from the two functions.





Get In Touch With Us



Two Park Ten Place
16300 Katy Freeway Suite 100
Houston, TX 77094



+1 281 599 3400



getstarted@openit.com



www.openit.com



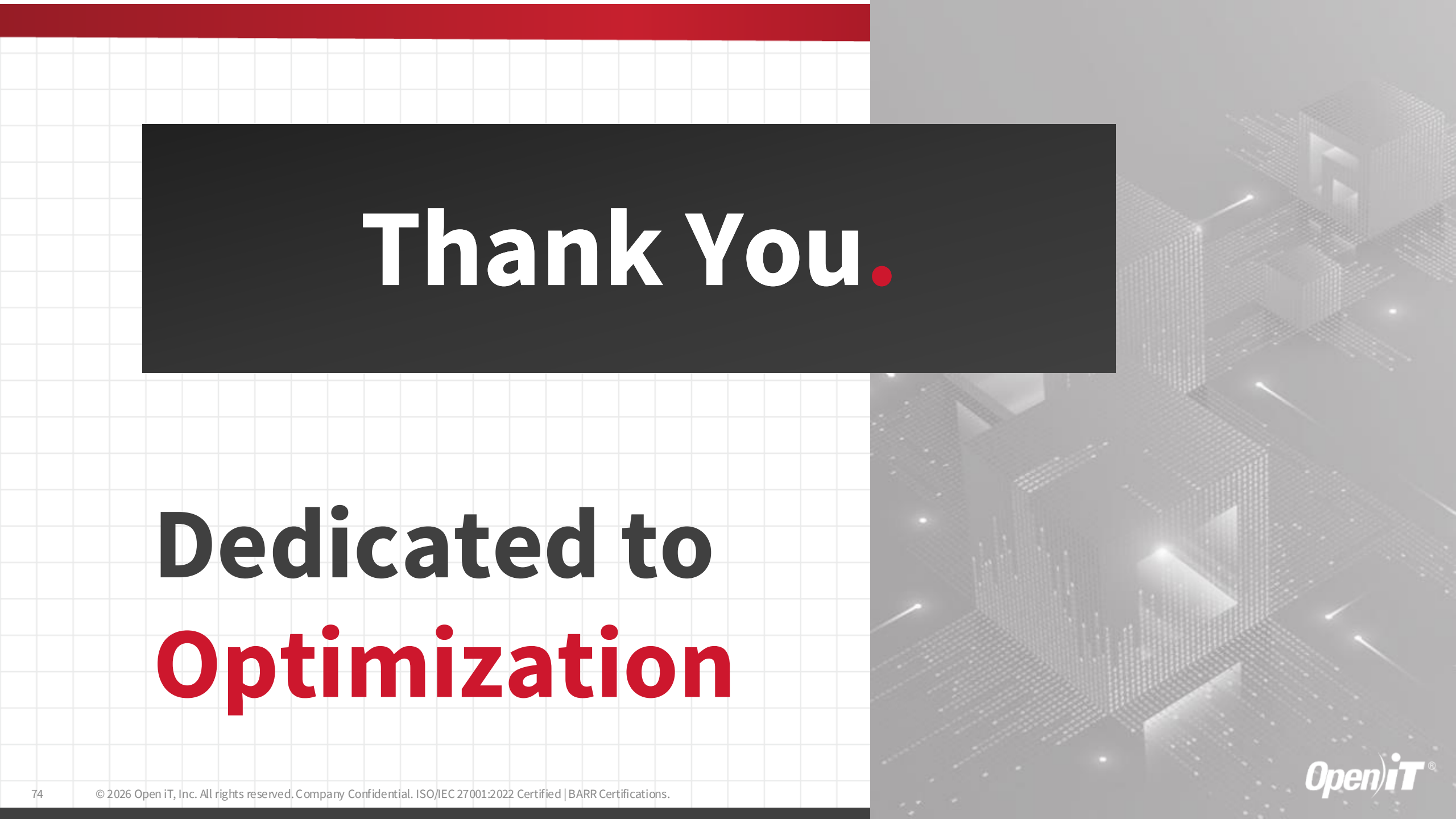
[/OpenITInc](https://www.linkedin.com/company/openit/)



[/OpenITInc](https://www.facebook.com/OpenITInc)



[@OpenITInc](https://www.youtube.com/channel/UCOpenITInc)



Thank You.

Dedicated to Optimization

